Nainisha Bandari

CHALLENGES & SOLUTIONS

1.API Security

Challenge

One of the major challenges during backend development was ensuring that all APIs were secure and protected from unauthorized access. Since the application manages user accounts, profile information, wardrobe data, wishlist items, cart details, and order information, exposing APIs without proper protection could lead to data breaches and malicious activities. Another concern was preventing unauthorized users from accessing administrative functionalities such as product management and inventory updates.

Solution

JWT (JSON Web Token) authentication was implemented to secure API endpoints and manage user sessions. Middleware was developed to verify tokens before granting access to protected routes. Passwords were encrypted using bcrypt hashing before storage in the database, ensuring that sensitive credentials remained protected even in the event of a database leak. Role-based authorization mechanisms were introduced to differentiate between regular users and administrators. Input validation was also applied using validation middleware to prevent invalid or malicious requests from reaching the server.

Outcome

- Improved application security.
- Protected sensitive user data.
- Reduced risks of unauthorized access.
- Established a scalable authentication framework.

2.Database Design

Challenge

Designing the database structure was challenging because the platform required multiple interconnected modules such as Users, Products, Wishlist, Cart, Orders, Measurements, Avatars, Wardrobe, and Try-On data. Maintaining efficient relationships between collections while ensuring high performance and scalability was a significant concern. Poor schema design could lead to redundant data, slower queries, and difficulties in future feature expansion.

Solution

MongoDB was selected due to its flexibility and ability to handle evolving data structures. Collections were designed using references and embedded documents where appropriate. Relationships between users and other entities such as orders, wishlists, and wardrobes were carefully planned to minimize duplication and improve retrieval efficiency. Proper indexing strategies were applied to frequently queried fields such as email addresses, product categories, and user identifiers.

Outcome

- Efficient data organization.
- Reduced redundancy.
- Faster query execution.
- Easier maintenance and scalability.

3.Image Upload Handling

Challenge

The application relies heavily on image management for profile pictures, product images, wardrobe uploads, avatar generation, and AI try-on outputs. Managing large image files directly on the server could increase storage requirements and negatively impact

application performance. Ensuring secure and reliable image uploads was another challenge.

Solution

Cloudinary was integrated as a cloud-based image management solution. Multer middleware was used to process multipart form-data uploads before forwarding files to Cloudinary. Uploaded images were automatically optimized, compressed, and stored in the cloud. Only image URLs were saved in MongoDB, reducing database storage requirements and improving overall efficiency.

Outcome

- Faster image delivery through CDN.
- Reduced server storage burden.
- Improved application performance.
- Enhanced media management capabilities.

4. Authentication & Session Management

Challenge

Managing secure user authentication while maintaining a smooth user experience was another challenge. The system needed to support traditional login, Google OAuth authentication, token-based sessions, and secure logout mechanisms.

Solution

JWT authentication and Google OAuth were integrated to provide multiple login options. Token verification middleware was implemented to secure private routes. Expiration policies and session validation mechanisms were designed to improve overall security.

Outcome

- Secure user authentication.
- Improved user convenience.
- Multiple login methods.

- Reliable session management.

5.Testing & Debugging

Challenge

Testing numerous API endpoints, database operations, authentication workflows, and image upload functionalities required significant effort. Identifying bugs during integration stages was particularly challenging.

Solution

Postman was extensively used to test API requests and responses. Various test scenarios were executed, including valid inputs, invalid inputs, authorization failures, and edge cases. Logs and debugging tools were used to monitor application behavior and resolve issues efficiently.

Outcome

- Reduced production errors.
- Improved application stability.
- Better API quality assurance.
- Increased system reliability.

6.Error Handling & Validation

Challenge

Handling unexpected user inputs, invalid requests, server failures, and database errors consistently across the application was essential. Without proper error handling, debugging and maintenance would become difficult.

Solution

Centralized error-handling middleware was implemented to capture and process application errors. Validation middleware ensured that incoming requests contained valid and complete data

before reaching business logic layers. Standardized error responses were used across all APIs.

Outcome

- Improved debugging process.
- Better API reliability.
- Consistent error responses.
- Enhanced user experience.

7.Scalability Planning

Challenge

The platform is expected to support increasing numbers of users, products, image uploads, and AI-generated content in the future. Designing the backend only for current requirements could create scalability issues later.

Solution

The backend was developed using a modular architecture with separate controllers, routes, services, and middleware. Cloud-based services such as MongoDB Atlas and Cloudinary were considered for handling large-scale data and media storage. API structures were designed to support future microservice migration if required.

Outcome

- Better scalability.
- Improved maintainability.
- Support for future feature expansion.
- Higher system reliability.